中南大学考试试卷（答案）

2010--2011 学年 2 学期 时间 100 分钟 2011 年 6 月 8 日

嵌入式系统 课程 32 学时 2.0 学分 考试形式： 开 卷

专业年级： 电气类 08 级 总分 100 分，占总评成绩 70 %

注：此页不作答题纸，请将答案写在答题纸上

一. 填空题 (每空 1 分，共 20 分)

1. 嵌入式系统的设计可以分成三个阶段：分析、 设计 ， 实现。
2. 微处理器有两种总线架构，使用数据和指令使用同一接口的是 冯诺依曼 ，分开的指令和数据接口、取指和数据访问可以并行进行的是 哈佛结构 。
3. ARM 核有两个指令集，分别是 ARM 、 THUMB
4. 嵌入性 ， 专用性， 与 计算机系统是嵌入式系统的 3 个基本要素。
5. 工业嵌入式系统的发展趋势是网路化、智能化和控制的分散化。 。
6. GPIO 口的功能是通过相应的 端口控制寄存器 来设定端口引脚的功能。
7. 看门狗定时器有三个控制其操作专用寄存器：分别为 WTCON 、 WTDAT 、 WTCNT 。
8. 端口 F 端口控制寄存器包括 GPFCON、 GPFDAT、 GPFUP。
9. ARM 公司提供的专门用于 ARM 相关应用开发和调试的综合性软件为： ADS1.2
10. DMA 的主要优点就是其传输数据不受 CPU 的干涉。

二. 简答题 (56 分，每小题 7 分)

1. 嵌入式开发环境主要包括那些组件

答：嵌入式系统开发需要交叉编译和在线调试的开发环境，主要包括

a. 宿主机
b. 目标机
c. 基于 Jtag 的 ICD 仿真器、或调试监控软件、或在线仿真器 ICE
d. 运行于宿主机的交叉编译器和链接器、以及开发工具链或软件开发环境
e. 嵌入式操作系统

2. ARM 核中什么寄存器用于存储 PC?R13 通常用来存储什么？ R14 通常用来存储什么？

答：R15 用于程序计数器 PC,R13 通常用来做堆栈指针寄存器， R14 通常用来做链接寄存器，保存函数调用的返回地址。

3. S3C2440A 中的中断控制器可以从 60 个中断源接收中断请求，简述中断控制器请求 ARM920T 的 FIR 或 IRQ 中断机制。

答：

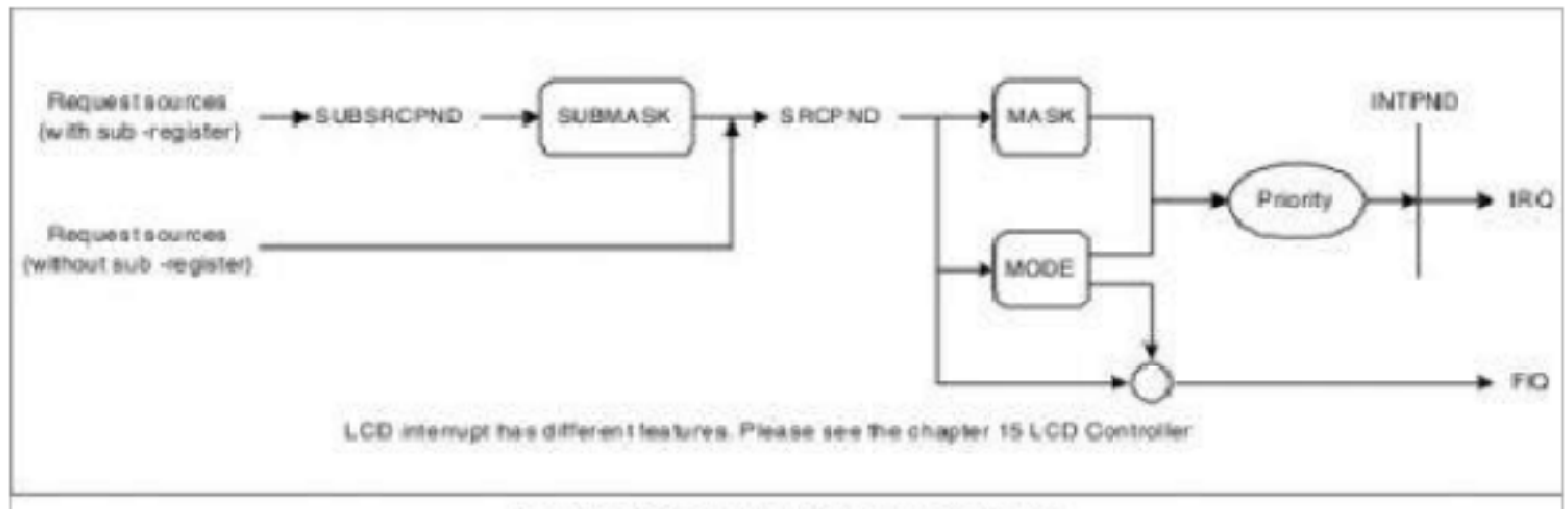


Figure 14-1. Interrupt Process Diagram

4. 简述嵌入式系统的概念、组成及特点。

答：嵌入式系统是以应用为中心， 以计算机技术为基础， 采用可裁减软硬件， 适用于对功能、可靠性、成本、体积、功耗等有严格要求的专用计算机系统。一般由嵌入式微处理器 、外围硬件设备、嵌入式操作系统以及用户的应用程序等四个部分组成。其特点有

1)嵌入式系统通常是面向特定应用的

2) 嵌入式系统是将先进的计算机技术、半导体技术和电子技术与各个行业的具体应用相结合后的产物

3)嵌入式系统的硬件和软件都必须高效率地设计， 量体裁衣、去除冗余

4)嵌入式系统和具体应用有机地结合在一起，它的升级换代也是和具体产品同步进行

5)为了提高执行速度和系统可靠性， 嵌入式系统中的软件一般都固化在存储器芯片或单片机本身中

6)嵌入式系统本身不具有自主开发能力。

5.试说明 看门狗 Watchdog 的作用和原理？

答：在嵌入式应用中， CPU 必须可靠工作，即使因为某种原因进入了一个错误状态，系统也应该可以自动恢复。看门狗的用途就是使微控制器在进入错误状态后的一定时间内复位。其工作原理是在系统正常工作时，用户程序每隔一段时间执行喂狗动作 ,如果系统出错，喂狗时隔超过看门狗溢出时间，那么看门狗将会产生复位信号，使系统复位。

6. 如果 S3C2440A 的 UART 时钟为 40MHz，要将 UART0 设为正常工作模式 ，并且为偶校验，停止位 1 位，数据位 7 位，波特率为 230400bps，该怎么设置， UART0 的线控制寄存器为 ULCON0，波特率设置寄存器为 UBRDIV0 。

答： UBRDIV0=0X09;

ULCON0&=0XFFFFFF00;

ULCON0|=0X2A;

7 在 C 语言与汇编程序混合编程中，子程序调用的 ATPCS 规定了哪些基本规则。简要说明寄存器使用规则。

答：基本规则有三个方面内容， 分别是寄存器的使用规则及其相应的名字， 数据栈的使用规则，参数传递规则。

A 寄存器的使用规则 ：

（1）子程序通过寄存器 R0~R3 来传递参数。 这时寄存器可以记作： A0~A3 ，被调用的子程序在返回前无需恢复寄存器 R0~R3 的内容。

（2）在子程序中，使用 R4~R11 来保存局部变量 ,这时寄存器 R4~R11 可以记作： V1~V8 。如果在子程序中使用到 V1~V8 的某些寄存器，子程序进入时必须保存这些寄存器的值，在返回前必须恢复这些寄存器的值， 对于子程序中没有用到的寄存器则不必执行这些操作。 在 THUMB 程序中，通常只能使用寄存器 R4~R7 来保存局部变量。

（3）寄存器 R12 用作子程序间 scratch 寄存器， 记作 ip；在子程序的连接代码段中经常会有

这种使用规则。

（4）寄存器 R13 用作数据栈指针， 记做 SP；在子程序中寄存器 R13 不能用做其他用途。 寄存器 SP 在进入子程序时的值和退出子程序时的值必须相等。

（5）寄存器 R14 用作连接寄存器， 记作 lr ；它用于保存子程序的返回地址， 如果在子程序中保存了返回地址，则 R14 可用作其它的用途。

（6）寄存器 R15 是程序计数器，记作 PC ；它不能用作其他用途。

（7） ATPCS 中的各寄存器在 ARM 编译器和汇编器中都是预定义的。

8. 结合实际说明一下 ARM 的中断进入和返回的过程。

答： A. 中断的进入：

（1）将下一条指令的地址存入相应连接寄存器 LR，以便程序在处理异常返回时能从正确的位置重新开始执行。

（2）将 CPSR 复制到相应的 SPSR 中。

（3）根据异常类型，强制设置 CPSR 的运行模式位。

（4）强制 PC 从相关的异常向量地址取下一条指令执行，从而跳转到相应的异常处理程序。也可以设置中断禁止位来阻止其他无法处理的异常嵌套。

B. 从中断返回：

（1）将链接寄存器 LR 的值减去相应的偏移量后送到 PC 中。

（2）将 SPSR 复制回 CPSR 中。

（3）如果进入时设置了中断禁止位，那么清除该标志。

三 程序设计（ 24 分，每小题 12 分）

1、通过控制 GPF0,GPF1,GPF2,GPF3 的控制实现对 LED1,LED2,LED3,LED4 轮流亮灭的控制。要求将程序缺失的部分补全。 （12 分）

```
AREA |DATA|,CODE,READONL    Y
        ENTRY
        ldr r13,=0x1000
        IMPORT LedMain
        b LedMain
        END
#define GPFCON (*(volatile unsigned *)0x56000050)
#define GPFDAT (*(volatile unsigned *)0x56000054)
#define GPFUP (*(volatile unsigned *)0x56000058)
int LedMain()
{

        void Delay(unsigned int);
        unsigned char ledtab[]={0xfe,0xfd,0xfb,0xf7};
        int i;
        GPFUP&= （0XFFFFFFF0）;            //使能上拉 F3-6
        GPFCON&= （0XFF00）;
        GPFCON|= （0X0055;）                  //将 F3-6 设为输出口
        while(1)
        {
                for(i=0;i<4;i++)
                {
```

```
                （GPFDA T=ledtab[i]; ）
                Delay(70);
            }
        }
        return(0);

}
```

```
void Delay(unsigned int x )
{
    unsigned int i,j,k;
    for(i=0;i<=x;i++)
        for(j=0;j<=0xff;j++)
            for(k=0;k<=0xff;k++);

}
```

2、 EINT0 接 S3C2440A 的 GPF0，并接上拉电阻，试设计主程序和中断服务程序。 (12 分)

```
Int main （）
{
Eint_int();
Enable_eint()
While(1)
{
Uart_printf(  “main is running  ”);
}
Void eint_int(void)
{Rgpfcon= Rgpfcon&~(3)|(1<<1);
Rgpfup|=(1<<0);
Rextint0=( Rextint0&~(7<<0))|(2<<0);
Pisr_eint0=(unsigned)einto_isr;
}
Void enable_eint(void)
Reintpend=0xffffff;
Reintmask=~(1<<11);
Rsrcpnd=bit_eint0;
Rintpnd=bit_eint0;
Rintmsk=~(bit_eint0);
}
Static void__eint0_isr(void)
{Clearpending(bit_eint0);
Uart_printf(  “eint0 is  running  );”
```